

or tampering. Use strong encryption protocols to protect data transmitted between the client and server.

- **HTTPS:** Enable HTTPS by installing an SSL/TLS certificate on your server. This encrypts data in transit, protecting it from eavesdropping and tampering. Here's an example configuration for Apache:

```
<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /path/to/
certificate.crt
    SSLCertificateKeyFile /path/to/
private.key
</VirtualHost>
```

- **End-to-end encryption:** For applications handling sensitive data, consider implementing end-to-end encryption (E2EE). This ensures that data is encrypted on the client-side before transmission and can only be decrypted by the intended recipient.

```
const crypto = require('crypto');

// Generate a key pair
const { publicKey, privateKey } =
crypto.generateKeyPairSync('rsa', {
  modulusLength: 2048,
  publicKeyEncoding: { type: 'spki',
format: 'pem' },
  privateKeyEncoding: { type: 'pkcs8',
format: 'pem' }
});

// Encrypt data using the public key
const data = 'Sensitive data to
encrypt';
const encryptedData = crypto.
publicEncrypt(publicKey, Buffer.
from(data, 'utf8'));

// Decrypt data using the private key
const decryptedData = crypto.
```

```
privateDecrypt(privateKey,
encryptedData);
```

```
console.log(decryptedData.
toString('utf8')); // Output: Sensitive
data to encrypt
```

Table 1 lists a few best strategies to enhance web server security.

Securing Apache web server

Securing an Apache web server involves implementing several best practices to protect against various security threats. Apache provides several security features that can be used to enhance the server's security. These include:

- **Access control:** Apache allows you to control access to directories and files using configuration directives like *Allow* and *Deny*.
- **Authentication:** It supports various authentication methods, such as basic authentication and digest authentication, to control access to resources.
- **Logging:** Apache's logging capabilities allow you to monitor and analyse server activity, which can help detect and respond to

security incidents.

How to configure Apache securely

It's essential to recognise that while Apache is inherently stable and secure, its level of security ultimately depends on how it's configured. Once Apache is installed, configuring the server to be as minimal as possible is key to enhancing its security posture.

Disable unused modules: Disable any Apache modules that are not needed for your server's operation. This reduces the attack surface and potential vulnerabilities.

```
# Disable modules
# LoadModule example_module modules/
mod_example.so
```

Use secure protocols: Configure Apache to use secure protocols like TLS/SSL to encrypt data in transit. Enable HTTPS by configuring SSL certificates.

```
# Enable SSL
SSLEngine on
SSLCertificateFile /path/to/certificate.
crt
```

Table 1: Strategies for enhancing web server security

Strategy	How to implement
Utilising firewall and intrusion detection/prevention systems	Install firewalls and IDS/IPS to monitor and filter traffic, blocking malicious activity.
Implementing a robust monitoring and logging system	Enable logging and monitoring to detect and respond to security incidents and unusual activity.
Conducting regular security audits and vulnerability assessments	Perform audits and assessments to identify and mitigate security vulnerabilities and risks.
Educating staff on security best practices	Provide training on security best practices to ensure employees understand and follow security policies.
Using security tools and services	Utilise antivirus software, web application firewalls, and other security tools to enhance protection.